

TRIỂN KHAI, PHÁT TRIỂN ỨNG DỤNG AI VÀ IOT

LAB 5 V4

DOCKERIZED MULTI-MODEL AI INFERENCE SERVICE FOR AIOT

Từ model AI gốc đến ONNX, API, giao diện upload ảnh và Docker deployment

Thông tin	Nội dung
Học phần	Triển khai, phát triển ứng dụng AI và IoT
Vị trí	Lab 5 trong chuỗi AIoT Intelligence & Deployment Pipeline
Trọng tâm	Đóng gói AI inference service bằng Docker; chạy local, Docker Desktop, WSL và Ubuntu server/VM
Model	Sensor inference demo và vision inference bằng model ảnh nhẹ SqueezeNet ONNX ImageNet-1K
Input	Telemetry JSON và ảnh upload qua giao diện web/API
Output	JSON response, ảnh annotated, top-k class, service logs, Docker image/container
Sản phẩm chính	Source code, Dockerfile, docker-compose.yml, hướng dẫn chạy và quan sát, ảnh minh chứng

Thông điệp chính: Lab 5 không tập trung train lại model. Nội dung chính là biến model đã có thành service có thể chạy, quan sát, kiểm thử và đóng gói lại bằng Docker.

I. Vị trí của Lab 5 trong chuỗi lab

Các lab trước đã hình thành dữ liệu, model và API cơ bản. Lab 5 chuyển trọng tâm sang triển khai: model đã có cần được đưa vào một service có endpoint rõ ràng, có log, có giao diện kiểm thử, có Dockerfile và có thể chạy lại trên môi trường khác.

Lab	Câu hỏi trung tâm	Sản phẩm
Lab 1	Hệ thống AIoT cần những khối nào?	Use-case, kiến trúc, dữ liệu, AI module
Lab 2	Dữ liệu đã sẵn sàng cho AI chưa?	Dataset sạch, feature, baseline API
Lab 3	Hệ thống có đang bất thường không?	anomaly score, severity, /detect-anomaly
Lab 4	Model dự báo tương lai như thế nào?	forecasting model, metrics, model card
Lab 5	Model đã có được đóng gói và chạy lại như thế nào?	AI inference service, Docker image/container, upload ảnh, logs

Câu chốt: Docker không làm model chính xác hơn. Docker giúp môi trường chạy model ổn định hơn, dễ chia sẻ hơn và dễ kiểm thử hơn.

II. Mục tiêu học tập

- Phân biệt train model và inference model.
- Mô tả được đường đi của model trong thực tế: framework gốc -> format triển khai -> API -> Docker image -> container.
- Phân biệt các định dạng model thường gặp: PyTorch, TensorFlow/Keras, scikit-learn, ONNX, TFLite và quantized model.
- Chạy được FastAPI service ở local trước khi Docker hóa.
- Upload ảnh qua giao diện web, quan sát ảnh kết quả có nhãn dự đoán và bảng top-k class.
- Build Docker image, chạy container bằng Docker Desktop GUI, WSL/Linux terminal và Docker Compose.
- Quan sát được logs, volume, port mapping và trạng thái container.
- Giải thích được lợi ích và giới hạn của Docker so với chạy trực tiếp bằng Python.
- Phân biệt Docker Desktop trên Windows, Docker trong WSL Ubuntu và Docker Engine trên Ubuntu server/VM.

III. Đường đi của model trong hệ thống thực tế

Không nên bắt đầu ngay bằng ONNX nếu chưa hiểu model gốc. Trong thực tế, model thường được huấn luyện bằng một framework như PyTorch, TensorFlow/Keras hoặc scikit-learn. Sau khi huấn luyện, model được lưu ở định dạng gốc, kiểm thử inference, sau đó mới cân nhắc chuyển sang định dạng phục vụ triển khai như ONNX, TFLite hoặc một biến thể đã lượng tử hóa.

```
Dữ liệu huấn luyện
-> train model trong PyTorch / TensorFlow / scikit-learn
-> lưu model ở định dạng gốc
-> kiểm thử inference local
-> chuyển sang ONNX / TFLite / quantized model nếu cần
-> đóng gói API service
-> build Docker image
-> chạy container
-> ghi log, giám sát, cập nhật version
```

Giai đoạn	Câu hỏi cần trả lời	Kết quả cần có
Train	Model học từ dữ liệu nào? Metric offline ra sao?	Model gốc và kết quả đánh giá
Save	Model được lưu bằng định dạng nào?	.pt/.pth, .keras/SavedModel, .joblib/.pkl
Convert	Có cần chuyển sang ONNX/TFLite không?	Model triển khai và báo cáo sai khác sau convert
Serve	Service nhận input/output như thế nào?	FastAPI endpoint, schema, response JSON

Containerize	Môi trường chạy được đóng gói chưa?	Dockerfile, image tag, container logs
Operate	Khi lỗi xảy ra thì kiểm tra ở đâu?	Health check, logs, version, rollback plan

Gợi mở: Nếu một model chỉ chạy được trong đúng máy huấn luyện ban đầu, model đó chưa sẵn sàng để triển khai. Cần xác định rõ format, runtime, dependencies, đường dẫn model, input schema và output schema.

IV. Định dạng model: từ framework gốc đến định dạng triển khai

Nguồn model	Định dạng thường gặp	Lợi ích	Đánh đổi
PyTorch	.pt, .pth, state_dict, TorchScript	Linh hoạt, phổ biến trong nghiên cứu deep learning	Khi load có thể cần class model Python và đúng version thư viện
TensorFlow/Keras	.keras, SavedModel, .h5	Phù hợp hệ TensorFlow, dễ chuyển sang TFLite	Runtime có thể nặng nếu chỉ cần inference nhẹ
scikit-learn	.joblib, .pkl	Gọn cho model ML cổ điển	Nhạy với version Python/sklearn
ONNX	.onnx	Định dạng mở, hỗ trợ inference bằng ONNX Runtime	Cần convert và kiểm tra operator/kết quả sau convert
TFLite	.tflite	Phù hợp mobile/edge	Có thể cần quantization; không phải operator nào cũng hỗ trợ tốt
Quantized model	int8/fp16 tùy runtime	Nhẹ hơn, có thể nhanh hơn	Có thể giảm accuracy; cần đánh giá lại

Câu hỏi kiểm tra: ONNX không phải là nơi bắt đầu của quá trình học máy. ONNX thường xuất hiện sau khi model đã được train và cần chuyển sang một định dạng thuận tiện hơn cho inference/deployment.

V. Service mẫu của Lab 5

Project mẫu là một FastAPI service có nhiều endpoint. Một phần endpoint xử lý telemetry JSON; một phần endpoint xử lý ảnh upload. Mục tiêu là cho thấy một inference service có thể phục vụ nhiều loại model và nhiều loại input.

Nhóm endpoint	Endpoint	Ý nghĩa quan sát
Trạng thái	/health	Kiểm tra service còn hoạt động không và vision model có load được không
Thông tin model	/model-info	Biết service đang dùng model nào, format nào và version nào
Telemetry	/detect-anomaly	Gọi luồng anomaly/risk kế thừa tính thần Lab 3
Telemetry	/forecast	Gọi luồng forecast/risk kế thừa tính thần Lab 4
Telemetry	/predict-risk	Chuyển output số thành risk level và recommendation

Nhóm vision	Endpoint	Kết quả trả về
Thông tin model ảnh	/vision/model-info	Tên model, input size, số class và trạng thái load model
API ảnh	/classify-image	JSON top-k class, confidence và inference time
API ảnh trực quan	/classify-image-annotated	Ảnh PNG có gắn nhãn dự đoán top-1
Giao diện web	/classify-image-demo	Ảnh gốc, ảnh kết quả, bảng top-k class và JSON response

VI. Hướng dẫn chạy và quan sát kết quả

Phần code có file hướng dẫn riêng: docs/HUONG_DAN_CHAY_VA_QUAN_SAT.md. File này trình bày từng bước chạy, kết quả cần nhìn thấy và ý nghĩa của kết quả. Không nên chỉ chạy lệnh; cần đọc log, mở endpoint, kiểm tra file output và ghi nhận trạng thái container.

Bước	Lệnh hoặc thao tác	Cần quan sát	Ý nghĩa
1	ls	Có app/, models/, outputs/, Dockerfile, docker-compose.yml	Nắm cấu trúc project trước khi chạy
2	pip install -r requirements.txt	Không lỗi import FastAPI/ONNX/Pillow	Môi trường local đủ thư viện
3	python scripts/download_vision_model.py	Có file squeezeNet1.1-7.onnx	Vision endpoint có model để inference
4	uvicorn app.main:app --reload	/health trả ok	API chạy local
5	Mở /classify-image-demo	Upload ảnh, có top-k class và ảnh annotated	Kiểm thử ảnh bằng giao diện
6	docker build -t lab5-aiot-inference:v4 .	Docker images có image v4	Đã đóng gói service thành image
7	Docker Desktop Run hoặc docker run	Container Running, logs có Uvicorn	Service chạy trong container
8	docker compose up --build	Compose chạy service	Chạy bằng file cấu hình thay lệnh dài

VII. Chạy local trước Docker

Chạy local là bước kiểm tra code và model trước khi container hóa. Nếu local đã lỗi, Docker sẽ làm tăng số điểm cần debug: package, path, port, model weights, volume và permissions.

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python scripts/download_vision_model.py
uvicorn app.main:app --reload
```

Đường dẫn	Cần thấy gì?	Nếu không đúng thì kiểm tra
http://127.0.0.1:8000/health	service status = ok	Server uvicorn có đang chạy không
http://127.0.0.1:8000/docs	Swagger UI hiển thị endpoint	FastAPI có import lỗi không
http://127.0.0.1:8000/classify-image-demo	Giao diện upload ảnh	Route /classify-image-demo có hoạt động không
outputs/vision_inference_log.csv	Có log sau khi upload ảnh	OUTPUT DIR và quyền ghi file

VIII. Giao diện upload ảnh

Giao diện /classify-image-demo giúp quan sát toàn bộ luồng vision inference mà không cần gõ curl. Ảnh được gửi lên API, backend tiền xử lý ảnh, gọi model ONNX, trả top-k class và tạo ảnh kết quả có nhãn dự đoán.

Thành phần giao diện	Vai trò	Cần phân tích
Ảnh gốc	Kiểm tra input đã upload đúng chưa	Ảnh có rõ chủ thể không?
Ảnh có nhãn	Hiển thị top-1 prediction trực quan	Nhãn có phù hợp bối cảnh ảnh không?
Bảng top-k	Hiển thị nhiều khả năng dự đoán	Top-1 và top-5 khác nhau thế nào?
Confidence	Ước lượng độ tin cậy tương đối của output	Confidence thấp có nên dùng để tự động ra quyết định không?
Inference time	Thời gian chạy model	Model nhẹ có lợi gì khi deploy?

Giới hạn model ảnh: SqueezeNet ImageNet-1K là classifier tổng quát. Model có thể nhận diện đồ vật phổ biến nhưng không thay thế model chuyên ngành như nhận diện bệnh cây, nhận diện lỗi sản phẩm hoặc giám sát an toàn lao động.

IX. Chạy Docker bằng Docker Desktop GUI

Docker Desktop phù hợp cho bước làm quen vì cho phép quan sát image, container, logs, port và volumes bằng giao diện. Trên Windows, Docker Desktop thường dùng WSL 2 backend để chạy Linux containers.

1. Mở Docker Desktop và kiểm tra Docker Engine đang Running.
2. Vào Settings -> General -> bật Use the WSL 2 based engine nếu dùng Windows.
3. Vào Settings -> Resources -> WSL Integration -> bật Ubuntu nếu dùng WSL.
4. Build image bằng lệnh: `docker build -t lab5-aiot-inference:v4`.
5. Mở tab Images và kiểm tra image lab5-aiot-inference:v4.
6. Bấm Run trên image, đặt container name lab5-aiot-api.
7. Map host port 8000 sang container port 8000.
8. Mount outputs vào `/app/outputs` và `models/vision` vào `/app/models/vision` nếu giao diện hỗ trợ.
9. Mở tab Containers để kiểm tra container Running.
10. Mở Logs để xem Uvicorn log.
11. Mở trình duyệt: `http://127.0.0.1:8000/classify-image-demo` và upload ảnh.

Minh chứng cần chụp: Images có image v4, Containers có container Running, Logs có Uvicorn, Swagger /docs, giao diện upload ảnh sau khi phân loại.

X. Chạy Docker bằng WSL/Linux terminal

WSL/Linux terminal giúp thao tác gần môi trường server hơn. Nếu dùng Windows, Docker CLI trong WSL thường kết nối tới engine do Docker Desktop quản lý khi WSL Integration được bật.

```
docker run --rm --name lab5-aiot-api -p 8000:8000 -v $(pwd)/outputs:/app/outputs -v $(pwd)/models/vision:/app/models/vision lab5-aiot-inference:v4
```

Thành phần lệnh	Ý nghĩa
<code>--name lab5-aiot-api</code>	Đặt tên container để dễ tìm trong logs và Docker Desktop
<code>-p 8000:8000</code>	Map port 8000 của host vào port 8000 của container
<code>-v \$(pwd)/outputs:/app/outputs</code>	Ghi logs từ container ra thư mục outputs trên host
<code>-v \$(pwd)/models/vision:/app/models/vision</code>	Cho container nhìn thấy model ONNX đã tải
<code>lab5-aiot-inference:v4</code>	Image được dùng để tạo container

XI. Docker Desktop, WSL Ubuntu và Ubuntu server khác nhau thế nào?

Môi trường	Bản chất	Phù hợp	Gần doanh nghiệp ở mức nào
Docker Desktop trên Windows	Ứng dụng desktop có GUI, dùng WSL 2 backend cho Linux containers	Học tập, demo, development trên laptop	Trung bình
Docker trong WSL Ubuntu	Gõ lệnh Docker trong Ubuntu WSL, thường kết nối Docker Desktop engine	Làm quen terminal Linux trên Windows	Khá gần
Docker Engine trên Ubuntu server/VM	Docker cài trực tiếp trên Ubuntu, không cần Docker Desktop GUI	Server, lab nội bộ, cloud VM, CI/CD	Gần thực tế triển khai hơn

Kết luận triển khai: Trong doanh nghiệp, server Linux thường không chạy Docker Desktop GUI. Hệ thống thường dùng Docker Engine, image registry, CI/CD, health check, logs và công cụ orchestration khi cần scale.

XII. Docker Compose và chuẩn doanh nghiệp

Docker Compose giúp mô tả cách chạy service bằng một file cấu hình. Với project nhiều service như API, dashboard, MQTT broker hoặc database, Compose giúp giảm lỗi so với nhiều lệnh docker run rời rạc.

```
docker compose up --build
docker compose ps
docker compose logs -f
docker compose down
```

Thành phần doanh nghiệp	Ý nghĩa trong triển khai AI service
Image tag	Quản lý phiên bản service/model, ví dụ v1.0, v1.1, rollback khi lỗi
Registry	Nơi lưu và phân phối image cho server/CI/CD
Health check	Kiểm tra service còn sống và model có load được không
Logs	Truy vết request, lỗi, latency, model response
Volume hoặc object storage	Lưu outputs, logs, model artifacts hoặc dữ liệu runtime
Monitoring	Theo dõi latency, lỗi, tài nguyên và drift nếu triển khai thật
Orchestration	Kubernetes hoặc nền tảng tương đương khi cần scale nhiều instance

XIII. So sánh không Docker và có Docker

Tiêu chí	Chạy local bằng Python	Chạy bằng Docker
Môi trường	Phụ thuộc Python/package trên máy	Được mô tả trong image
Debug code	Nhanh và trực tiếp	Cần đọc container logs
Chia sẻ cho máy khác	Dễ lỗi version/path/package	Dễ chạy lại nếu image và volume đúng
Đường dẫn model	Theo filesystem local	Theo đường dẫn trong container hoặc volume mount
Ghi log	Ghi trực tiếp vào thư mục local	Cần mount volume để giữ log sau khi xóa container
Gần triển khai thật	Thấp hơn	Cao hơn, nhất là với Docker Engine/Compose

Thao tác bắt buộc: Hoàn thành bảng so sánh sau khi chạy cả hai luồng. Kết luận cần dựa trên quan sát: API có chạy không, model có load không, log nằm ở đâu, container có restart được không.

XIV. Lỗi thường gặp và cách quan sát

Hiện tượng	Cần kiểm tra	Công cụ quan sát
Không mở được API	Container có chạy không, port mapping đúng không	docker ps, Docker Desktop Containers, Logs
Vision model chưa load	File ONNX và labels có trong models/vision không	ls models/vision, /vision/model-info
Upload ảnh lỗi	Định dạng/kích thước ảnh, python-multipart	Swagger response, browser console nếu cần
Không có log CSV	Volume outputs đã mount chưa, quyền ghi file	outputs/, docker inspect
Build chậm hoặc lỗi	Mạng, pip package, Docker cache	docker build logs
Local chạy được nhưng Docker lỗi	Dockerfile COPY, working directory, env var, volume	Dockerfile, logs, /health

XV. Nhiệm vụ thực hành

Mức	Yêu cầu	Sản phẩm
Bắt buộc	Chạy local, tải model, test /health, /docs, /classify-image-demo	Ảnh minh chứng local và log output
Bắt buộc	Build Docker image, chạy container, test lại upload ảnh trong Docker	Ảnh Docker Desktop/terminal và

		kết quả upload ảnh
Khá	Chạy bằng Docker Compose, đọc logs, giải thích port/volume	docker-compose.yml chạy được và ảnh logs
Giỏi	Thử thay đổi host port, đổi model path, hoặc chạy trên Ubuntu VM/server	Báo cáo lỗi gặp và cách xử lý

XVI. Câu hỏi phân tích sau lab

12. Vì sao cần chạy local trước khi Docker hóa?
13. Train model khác inference model ở điểm nào?
14. Model PyTorch/TensorFlow/scikit-learn thường được lưu bằng định dạng nào?
15. ONNX giải quyết vấn đề gì trong triển khai?
16. TFLite hoặc quantized model phù hợp trong tình huống nào?
17. Vì sao API nên trả top-k class thay vì chỉ top-1?
18. Confidence thấp gợi ý điều gì khi đọc kết quả model ảnh?
19. Docker image khác container ở điểm nào?
20. Vì sao cần port mapping 8000:8000?
21. Vì sao cần volume khi ghi log?
22. Docker Desktop khác Docker Engine trên Ubuntu server ở điểm nào?
23. Nếu container chạy nhưng trình duyệt không mở API được, cần kiểm tra những gì?
24. Trong doanh nghiệp, vì sao cần image tag, model version và health check?

XVII. Rubric đánh giá

Tiêu chí	Mô tả	Điểm
Hiểu đường đi model	Giải thích được framework gốc, định dạng triển khai, ONNX/TFLite và inference service	1.5
Chạy local	Cài môi trường, tải model, chạy API, test /health và /docs	1.0
Giao diện upload ảnh	Upload ảnh, đọc top-k class, confidence, ảnh annotated và log	1.5
Dockerfile/image	Build được image, hiểu COPY/RUN/CMD/EXPOSE	1.0
Docker Desktop GUI	Quan sát image/container/logs, chạy được service bằng giao diện hoặc kết hợp terminal	1.0
WSL/Linux terminal	Chạy docker run với port và volume đúng	1.0
Docker Compose	Chạy docker compose up --build và đọc logs	1.0
So sánh local và Docker	Có bảng quan sát và nhận xét đúng bản chất	1.0
Phân tích triển khai	Nêu được health check, logs, version, registry, edge/cloud và giới hạn model ảnh	1.0
Tổng		10.0

XVIII. Sản phẩm nộp

- Source code project.
- Ảnh /health khi chạy local.
- Ảnh /classify-image-demo sau khi upload ảnh local.
- Ảnh Docker Desktop tab Images có image lab5-aiot-inference:v4.
- Ảnh Docker Desktop tab Containers có container lab5-aiot-api đang Running.
- Ảnh Docker Desktop Logs.
- Ảnh Swagger /docs khi chạy trong container.
- Ảnh /classify-image-demo khi chạy trong container.
- File outputs/service_log.csv hoặc outputs/vision_inference_log.csv.
- Bảng so sánh chạy local và Docker.

- Trả lời câu hỏi phân tích sau lab.

Tài liệu tham khảo

- Docker Desktop WSL 2 backend: <https://docs.docker.com/desktop/features/wsl/>
- Docker Engine trên Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
- Docker Compose documentation: <https://docs.docker.com/compose/>
- FastAPI file upload: <https://fastapi.tiangolo.com/tutorial/request-files/>
- PyTorch save/load model tutorial: https://docs.pytorch.org/tutorials/beginner/basics/saveloadrun_tutorial.html
- TensorFlow Save and load models: https://www.tensorflow.org/tutorials/keras/save_and_load
- ONNX overview: <https://onnx.ai/>